

AIM PROTOCOL

Agri • Identity • Microfinance

Version 1.1 — Release Notes & Technical Reference

Built by Amjad Kamara
Founder, Aadios Systems (SL) Ltd. • Sierra Leone
Solana Frontier Hackathon 2026

Live: <https://aim-protocol.vercel.app/>

Status	Live on Solana Devnet
Frontend Repo	github.com/amjadkamara/aim-protocol
Smart Contract Repo	github.com/amjadkamara/aim-program
Program ID	AhHHJTU5vodDYE2yLNet2bE6jad9F3xSfbLQdUmykKqB
Hackathon	Solana Frontier 2026 — V1.1 Submission
Network	Solana Devnet

1. Project Overview

AIM Protocol is a decentralized microfinance platform built on the Solana blockchain. It enables smallholder farmers in Sierra Leone and across Africa to establish a verifiable on-chain identity and access crop-backed microloans — without a bank account, collateral, or paperwork.

Version 1.1 was built and submitted during the Solana Frontier Hackathon 2026 over approximately four weeks, starting from zero Solana knowledge and completing with a fully deployed application running real smart contracts on Solana devnet.

1.1 What Was Built

- React + Vite frontend deployed on Vercel at aim-protocol.vercel.app

- Phantom wallet connection via Solana Wallet Adapter
- On-chain Farmer ID registration via an Anchor smart contract — one wallet, one identity
- Crop-backed microloan disbursement gated by verified Farmer ID
- Active loan enforcement — a second loan is blocked while one is open
- Real on-chain transactions fully verifiable on Solana Explorer
- Professional UI with persistent layout, stats bar, and CTA sections

2. The Problem AIM Protocol Solves

Bo, Sierra Leone was once surrounded by productive farmland that fed the city at affordable prices. Today those farms are largely idle — not because farmers stopped wanting to farm, but because without access to affordable credit they cannot invest in quality seeds, fertiliser, or equipment at the start of each planting season.

Poor yields make farming look unprofitable compared to urban wages. The younger generation leaves for cities. Traders travel further into the interior to source food, adding transport costs to every market item. The result: a region surrounded by farmland paying elevated prices for food that should be local and inexpensive.

The root cause is a credit gap. Traditional lenders require collateral that smallholder farmers do not have, application processes that take weeks, and interest rates of 30-60% annually — terms that make agricultural investment economically unviable.

2.1 How Blockchain Addresses This

Traditional Barrier	AIM Protocol Solution
Bank account required	Wallet address is the identity — anyone with a smartphone can participate
Collateral documentation	On-chain Farmer ID replaces collateral — identity is the credential
Loan officer approval (weeks)	Smart contract enforces rules automatically — loan issued in minutes
No credit history	Every transaction recorded permanently on-chain — credit record builds over time
High fixed fees	Solana transaction costs under \$0.001 — sub-\$500 loans are economically viable

3. Technology Stack

3.1 Frontend

Technology	Version / Notes
React	JavaScript UI framework — component-based architecture
Vite	Build tool and development server — fast hot module replacement
Tailwind CSS	v4 — utility-first CSS framework, configured via Vite plugin
Lucide React	Icon library
Solana Wallet Adapter	React library for Phantom wallet connection and state management

3.2 Blockchain / Smart Contract

Technology	Version / Notes
Solana	Blockchain — high throughput, sub-cent transaction costs
Rust	Smart contract language — compiled to BPF bytecode for Solana runtime
Anchor Framework	0.31.1 — structured smart contract development for Solana
@coral-xyz/anchor	JavaScript/TypeScript client for interacting with Anchor programs
Solana Web3.js	Low-level JavaScript library for Solana RPC interactions
Phantom Wallet	Browser extension wallet — devnet and mainnet

3.3 Infrastructure

Tool	Purpose
Vercel	Frontend hosting — auto-deploys from GitHub on every push
Solana Devnet	Test network — real smart contract behaviour, no financial risk
GitHub	Two repositories: aim-protocol (frontend), aim-program (smart contract)
WSL Ubuntu	Linux environment on Windows — required for Solana CLI and Anchor toolchain
VS Code	Development environment

4. Smart Contract — Rust + Anchor

The smart contract is the trust layer of AIM Protocol. Deployed on Solana devnet, it enforces all business rules automatically — no human intermediary required. Once deployed, its logic cannot be changed without redeployment under a new Program ID.

4.1 Program Instructions

Instruction	Description	Key Constraint
create_farmer_id	Creates a FarmerAccount storing: owner wallet, full name, crop type, district, farm size, loan status, registration timestamp	One registration per wallet — duplicate attempts fail
request_loan	Creates a LoanAccount storing: amount, purpose, repayment weeks, status, timestamp	ActiveLoanExists error returned if farmer already has an open loan
repay_loan	Marks loan as repaid and clears the farmer's active loan flag	Farmer becomes eligible to borrow again immediately

4.2 Account Structures

Account	Fields Stored
FarmerAccount	owner (Pubkey), full_name, crop_type, district, farm_size (f64), has_active_loan (bool), created_at (i64)
LoanAccount	farmer (Pubkey), amount (u64 lamports), purpose, repayment_weeks, is_repaid (bool), created_at (i64)

V1.1 Limitation — Keypair-Based Accounts:

In V1.1, farmer and loan accounts were created using randomly generated keypairs rather than Program Derived Addresses (PDAs). This meant the frontend had to generate and store a keypair locally, which was fragile and insecure. V2.1 replaced this entirely with PDA enforcement — one wallet deterministically maps to one FarmerAccount.

5. Frontend Architecture

5.1 Component Structure

File	Purpose
src/main.jsx	Entry point — wraps app in ConnectionProvider, WalletProvider, WalletModalProvider
src/App.jsx	Page routing via React state (home, farmerid, microloan)
src/Layout.jsx	Persistent navbar and footer wrapper — renders across all pages
src/FarmerID.jsx	Farmer ID registration form and success screen
src/Microloan.jsx	Microloan request form, approval, and active loan state
src/useAimProgram.js	Custom hook — AnchorProvider setup, createFarmerID and requestLoan functions
src/idl/aim_program.json	Anchor-generated IDL — bridge between JavaScript frontend and Rust contract

5.2 Wallet Integration

The Solana Wallet Adapter library manages wallet connection state. Three providers wrap the application in main.jsx: ConnectionProvider (RPC endpoint), WalletProvider (wallet state), and WalletModalProvider (wallet selection modal). The useWallet hook exposes connected, publicKey, and signTransaction throughout all components.

5.3 Smart Contract Integration

A custom hook (useAimProgram.js) creates an AnchorProvider from the live RPC connection and connected wallet, then instantiates the Program using the deployed IDL and Program ID. This exposes createFarmerID() and requestLoan() as async functions that build, sign, and submit Anchor transactions via Phantom.

6. The Full User Flow

Step	Action	On-Chain Result
1	User visits aim-protocol.vercel.app and connects Phantom wallet	Wallet public key available to application — no on-chain action
2	User fills Farmer ID form (name, crop, district, farm size) and submits	FarmerAccount created on Solana devnet — transaction verifiable on Explorer
3	User fills microloan request (purpose, amount, repayment period) and submits	LoanAccount created — has_active_loan set to true on FarmerAccount
4	User attempts second loan while first is active	Smart contract returns ActiveLoanExists

		error — frontend displays plain English message
5	User repays loan	LoanAccount marked repaid — has_active_loan cleared — farmer eligible to re-borrow

7. Deployment

Component	Platform	Details
Frontend	Vercel	Auto-deploys from GitHub main branch on every push. Live at aim-protocol.vercel.app within 2 minutes of each commit.
Smart Contract	Solana Devnet	Program ID: AhHHJTU5vodDYE2yLNet2bE6jad9F3xSfbLQdUmykKqB. Deployed via Anchor CLI from WSL Ubuntu.
IDL	Bundled with frontend	aim_program.json generated by anchor build — copied to src/idl/ and committed to the frontend repository.

8. Repository Structure

Repository	Contents	Deployment Target
aim-protocol	React + Vite frontend — all UI components, wallet integration, useAimProgram hook, IDL	Vercel
aim-program	Rust + Anchor smart contract — lib.rs, Anchor.toml, Cargo.toml, test suite	Solana Devnet

Separating frontend and smart contract into distinct repositories is standard practice in Web3 development. The two components have different deployment targets, different build toolchains, and different development cycles. The IDL file serves as the shared interface between them.

9. Known Limitations in V1.1

Limitation	Impact	Resolution in V2.x
Keypair-based accounts — not PDAs	Frontend stored keypairs locally — fragile, insecure, no duplicate prevention	V2.1 — PDA enforcement throughout

No loan repayment UI	Farmers could not close loans or re-borrow	V2.1 — RepayLoan screen with full lifecycle
Repaid loan PDA blocked re-borrowing	Critical bug — loan cycle broken after first repayment	V2.1 — close constraint on repayment frees PDA
Raw Solana errors shown to users	custom program error: 0x0 — meaningless to farmers	V2.1 — parseAnchorError() helper with plain English
No loan status visibility	Farmers had no dashboard to track position	V2.1 — Loan Status Dashboard
No loan history	No borrowing record — core fintech requirement	V2.1 — On-chain transaction history via RPC
No SOL or network warnings	New users silently failed with 0 SOL on wrong network	V2.1 — WalletStatus banner with devnet and balance detection

10. Version 2 Roadmap

V1.1 proved the concept — a farmer can register a verified identity on-chain and receive a microloan in a single session, without a bank account or paperwork. V2.x builds production-quality infrastructure on this foundation.

Smart Contract

- PDA enforcement — one wallet = one Farmer ID, blockchain-enforced
- Full loan lifecycle — repayment instruction with PDA closure and re-borrow capability
- Credit scoring based on repayment history — stored in FarmerAccount
- Multi-lender architecture — LenderAccount PDAs, admin-approved lender registration
- Loan counter per farmer for history indexing across lenders
- USDC stablecoin disbursement replacing SOL — removes price volatility
- Oracle integration for crop price feeds

Frontend

- Admin dashboard — live farmer and loan monitoring from devnet
- Authenticated home state — personalised greeting for registered farmers
- Public farmer profile lookup by wallet address
- Loan marketplace — browse approved lenders and their terms
- Lender registration flow — multi-step onboarding for organisations
- KYC integration — government ID verification for donor compliance

Infrastructure

- Mainnet migration following security audit
- Custom domain via Aadios Systems
- Analytics and monitoring

Business

- Pilot with one Sierra Leone farming cooperative — 10-20 farmers on devnet
- Superteam Africa chapter application
- Grant applications: IFAD, World Bank, Solana Foundation African Ecosystem
- Aadios Systems platform launch

AIM Protocol V1.1 — Proof of concept delivered. Africa on Solana.

Amjad Kamara | Aadios Systems (SL) Ltd. | <https://aim-protocol.vercel.app>
Built from Sierra Leone